

OBJECTIVES

- Master more concepts/technologies related to K8
- Students to start working on the UI using React

VALIDATING LAB 5 PROGRESS

Students will be demonstrating that they have completed fully lab 5 and it's working on their computer.

INDIVIDUAL EXERCISES

Task / Exercise C1 [60 Minutes] (DMS Services inside K8) :

Students need to integrate all the services created within Spring Boot to work inside Kubernetes. This includes : Documents, Comments, Gateway. The comments service needs to use its own postgres volume/instance. (no need for the ESB)

Change the architecture for getting a full document as :

1. client calls Gateway
2. Gateway routes or forwards to Documents
3. Documents calls Comments

For adding a comment, make the flow as : Gateway -> Comments

Task / Exercise C3 [15 Minutes] (Hello World UI in React)

Students should :

- Set up their dev environment to create the hello world ui using React.
- Create the dms internal dashboard.

OPTIONAL EXERCISES

Task / Exercise P1 (Tracing using Jaeger) :

In this exercise, you will observe how a request travels across multiple microservices using distributed tracing. You will deploy Jaeger as a pod inside your Minikube cluster, instrument your three DMS Spring Boot services with the OpenTelemetry Java Agent, and then send a real request through the Gateway and observe its full trace waterfall in the Jaeger UI.

1. **Deploy Jaeger in Minikube**
 - a. Deploy Jaeger as a Kubernetes pod.
 - b. Expose the Jaeger UI and access it in your browser.
2. **Enable tracing in the services**
 - a. Add OpenTelemetry instrumentation to the Spring Boot services (Gateway, Documents, and Comments).
 - b. Configure the services to export traces to Jaeger using the OTLP endpoint.
3. **Generate a distributed request**
 - a. Send a request through the Gateway that triggers calls to the Documents service and the Comments service.
4. **Analyze the trace**
 - a. Open the Jaeger UI.
 - b. Locate the generated trace.
 - c. Observe the trace waterfall and identify the sequence of service calls and their execution time.

Task / Exercise P2 :

In this exercise, you will collect and analyze logs from Kubernetes services using **Loki** and **Grafana**.

1. Deploy the logging stack

- Use **Helm** to deploy a lightweight **Loki + Grafana** stack in Minikube.
- Verify that both Loki and Grafana pods are running.
- Access the Grafana dashboard from your browser.

2. Collect logs from Kubernetes pods

- Configure log collection so that **pod logs are sent to Loki** (e.g., using Promtail).
- Ensure that logs from the **Documents service pod** are visible in Grafana.

3. Explore the logs

- In Grafana, open the **Explore** section.
- Select **Loki** as the data source and inspect logs from the running services.

4. Query logs using LogQL

- Write a **LogQL query** that filters logs from the **Documents service**.
- Modify the query to display **only error messages**.

NEXT DELIVERABLE : 11 APRIL 2026

- FRONT-END PROTOTYPE SUBMISSION FOR THE DMS : GROUP PROJECT.
 - *React App showing the UI components connected to Fake APIs.*